

ECE391
Computer System Engineering
Lecture 7

University of Illinois at Urbana-Champaign

Spring 2024

Lecture Topics

- Multiprocessors and locks
- Spin locks

Recap

- Interrupts are awesome!
- Use same mechanism for exceptions and syscalls

The diagram illustrates the structure of the Interrupt Descriptor Table (IDT) and how it is used to find the address of an interrupt handler. At the top, a horizontal bar represents the IDT's base and limit. The left portion is labeled "IDT BASE ADDRESS" with bit positions 47 and 16 indicated above it. The right portion is labeled "IDT LIMIT" with bit positions 15 and 0 indicated above it. Below this bar, a vertical line represents the IDT array. An arrow points from the "IDT BASE ADDRESS" field to a circular adder (containing a "+"). Another arrow points from the "IDT LIMIT" field to the same adder. The output of the adder points to the first entry in the "INTERRUPT DESCRIPTOR TABLE". The table is shown as a vertical stack of four entries, each labeled "GATE FOR INTERRUPT #N", "GATE FOR INTERRUPT #3", "GATE FOR INTERRUPT #2", and "GATE FOR INTERRUPT #1" from top to bottom. An arrow points from the "IDT BASE ADDRESS" field to the first entry in the table.

31	2	2	2	2	2	1	1	1	1	1	1	1	1	1	1	0					
	4	3	2	1	0	9	8	7	6	5	4	3	2	1	0	9	8	7			
OFFSET 31:16										P	D P L	0	1	1	1	0	0	0	0	RSVRD.	+4
SEGMENT SELECTOR										OFFEST 15:00										+0	

31		1 1 1 1 1 1 1 1 6 5 4 3 2 1 0 9 8 7 6 5 4															0
OFFSET 31:16		P	D P L	0 1 1 1 1					0 0 0			RSVRD.		+4			
SEGMENT SELECTOR		OFFEST 15:00													+0		

Precap

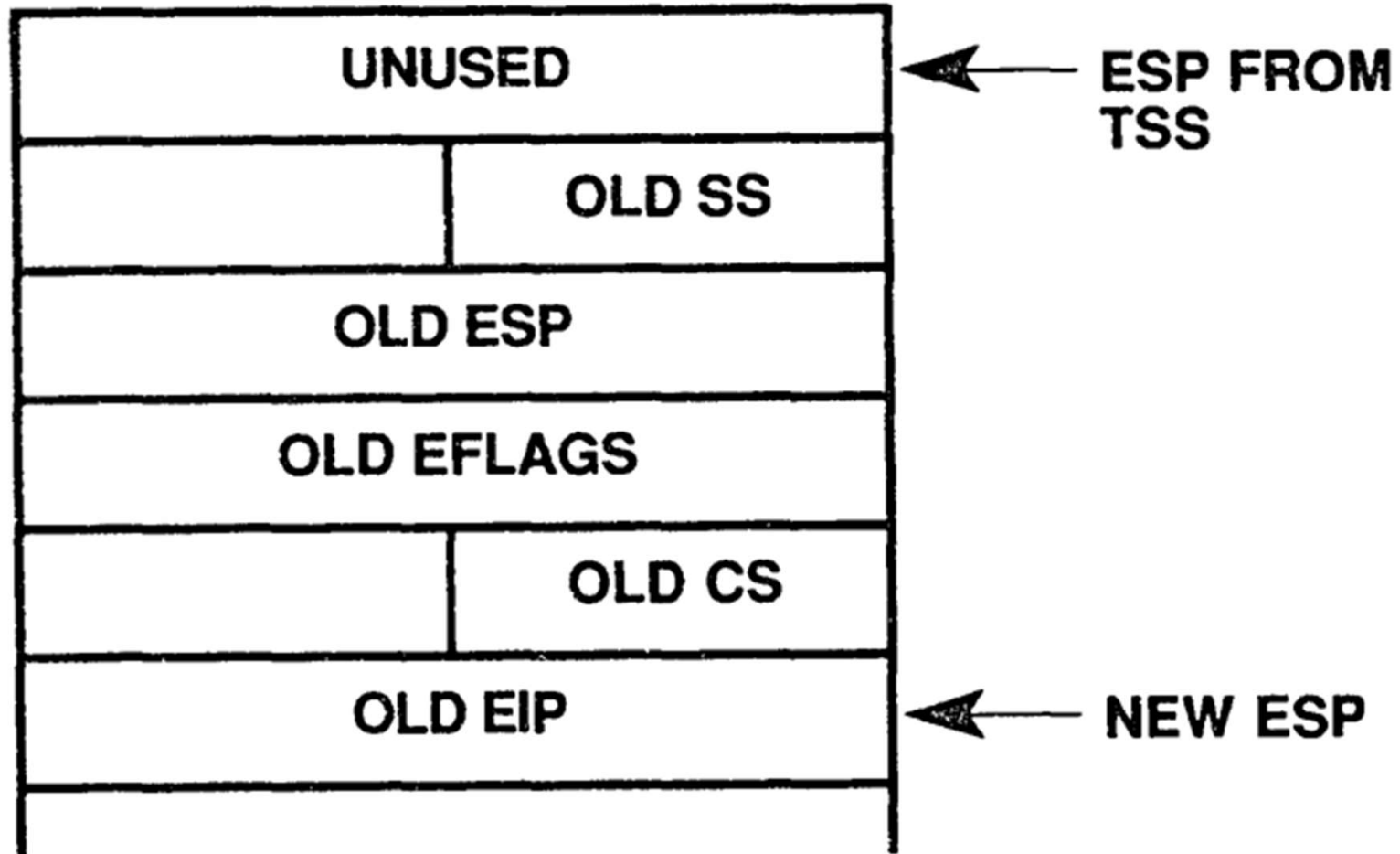
- Interrupts are awesome!
- Use same mechanism for exceptions and syscalls
- Hardware interrupts can happen at any time
- Need to protect critical sections
 - State-sensitive device interactions
 - Operations on data structures
- We do this by disabling interrupts in critical section
 - CLI to disable, STI to enable
 - Better: `safe EFLAGS, CLI, (critical section), restore EFLAGS`
 - Best: `local_irq_save` and `local_irq_restore` macros

Execution Context



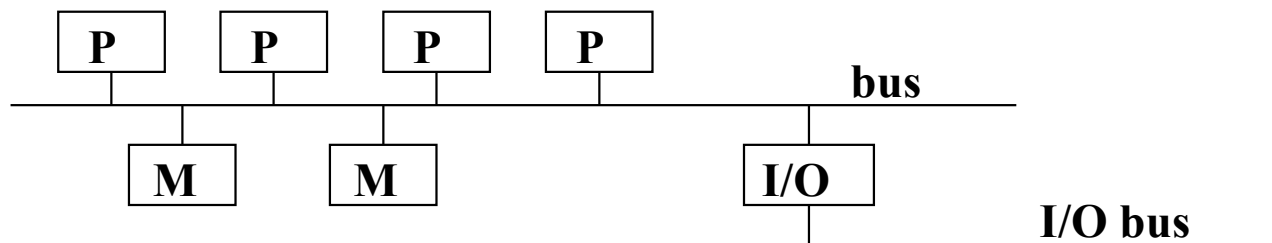
- Process context:
- Interrupt context:

Stack on Entry to Syscall Handler



Multiprocessors and Locks

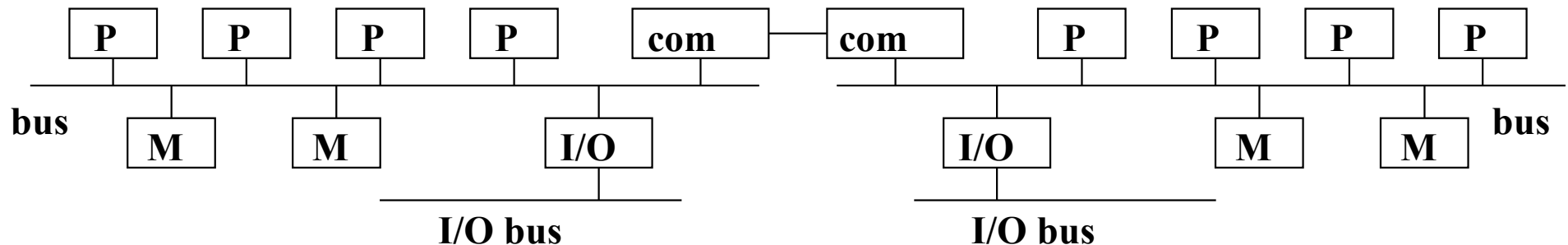
- We solved the critical section problem for uniprocessors
- What about multiprocessors?
 - *CLI ... critical section STI*
- What is a multiprocessor?
 - usually a symmetric multiprocessor (SMP)



- symmetric aspect: all processors have equal latency to all memory banks
- multicore processors are similar from our perspective

Multiprocessors and Locks (cont.)

- Some non-uniform memory architecture (NUMA) machines were built



Back to Serial Ports

- What happens if two CPUs call into `getchar()` at the same time?
- Both CPUs will try to update same data structure (ring buffer)
- Instructions from each CPU interleaved arbitrarily
- Need to protect critical sections from other CPUs

```
void rxdata(void) {
    int tpos = (rxhpos + rxcnt) % 4;
    if (rxcnt < 4) {
        // read byte from RBR
        rxbuf[tpos] = inb(0x3F8);
        rxcnt = rxcnt + 1;
    } else {
        // receiver buffer overflow
    }
}

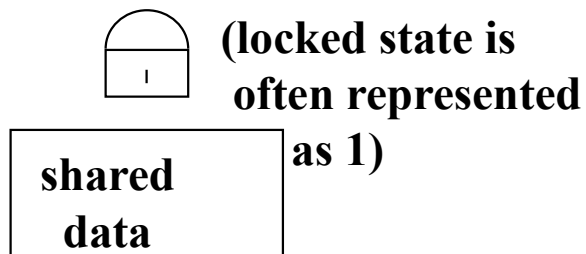
char getchar(void) {
    char c;
    while (rxcnt == 0) {
        // wait for data to become available
    }
    c = rxbuf[rxhpos];
    rxhpos = (rxhpos + 1) % 4;
    rxcnt = rxcnt - 1;
    return c;
}
```

Multiprocessors and Locks (cont.)

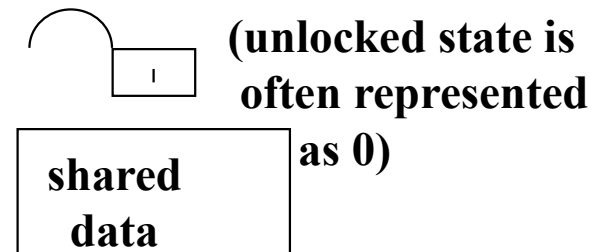
- Multithreaded code is not protected by *IF*
- Why haven't we solved the atomicity problem on multiprocessors?
 - interrupts **are masked** if *IF* is cleared!
 - answer: *IF* is **not cleared** on other processors!
 - just tell other processors to clear IF, too?
 - too slow
 - requires an interrupt!
- We need to use shared memory to synchronize...

Multiprocessors and Locks (cont.)

- Logically, we use a lock
 - when we want to access a piece of shared data we first look at the lock
 - if it's locked, we wait until it's unlocked

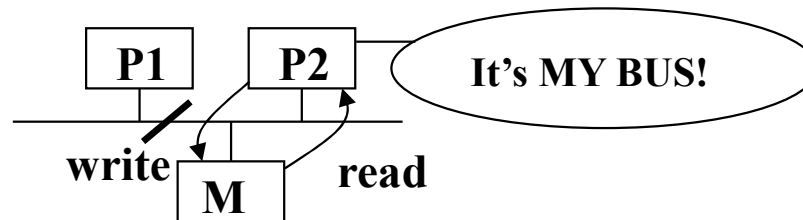


- once it's unlocked
 - we lock it
 - access the data
 - then unlock it



Multiprocessors and Locks (cont.)

- Locking must be atomic with respect to other processors!



INCL (%EAX)



read (%EAX)

add 1

put it back

LOCK: Prefix - execute following instruction with bus locked

LOCK INCL (%EAX)

An Example of Locking Implementation

```
/* The caller defines int lock. Calling TestAndSet (&lock) sets lock to 1 and returns the old
value of lock. If lock is 0 then TestAndSet(&lock) returns 0 and sets the lock to 1. If the
lock is already set to 1 by another process or thread, then 1 is returned. The caller keeps
retrying TestAndSet(&lock) until it returns 0. */
```

TestAndSet:

.....

```
pushl %ebx
```

```
movl 8(%ebp), %ebx      # &lock to ebx
                        # 1 (true) to eax
```

```
#Atomically exchange eax with lock.
```

```
#The atomicity of xchg guarantees that at most
```

```
#one thread holds the lock at any point in time
```

```
popl %ebx
```

.....

```
ret                    #return value (old value of lock) is already in eax
```

xchg => exchanges the contents of a register with the contents of any other register or memory location

Spin Locks

- The simplest lock
 - spin lock
 - lock op
 - other work to do? ignore it!
 - spin in a tight loop on the lock (hence the name)
- Once successful, program/interrupt handler **owns** the lock

Spin Locks (cont.)

- Only the owner can unlock
 - How?
 -
 - Need to be atomic?
 -

Linux Spin Lock API

- Static initialization

```
static spinlock_t a_lock = SPIN_LOCK_UNLOCKED;
```

- Dynamic initialization

```
spin_lock_init (&a_lock);
```

- When is dynamic initialization safe?
 - lock must not be in use (race condition!)
 - other synchronization method must prevent use

Linux Spin Lock API – Basic Functions

```
void spin_lock (spinlock_t* lock);
```

```
void spin_unlock (spinlock_t* lock);
```

Linux Spin Lock API – Testing Functions

```
int spin_is_locked (spinlock_t* lock);
```

returns 1 if held, 0 if not, but beware of races!

```
int spin_trylock (spinlock_t* lock);
```

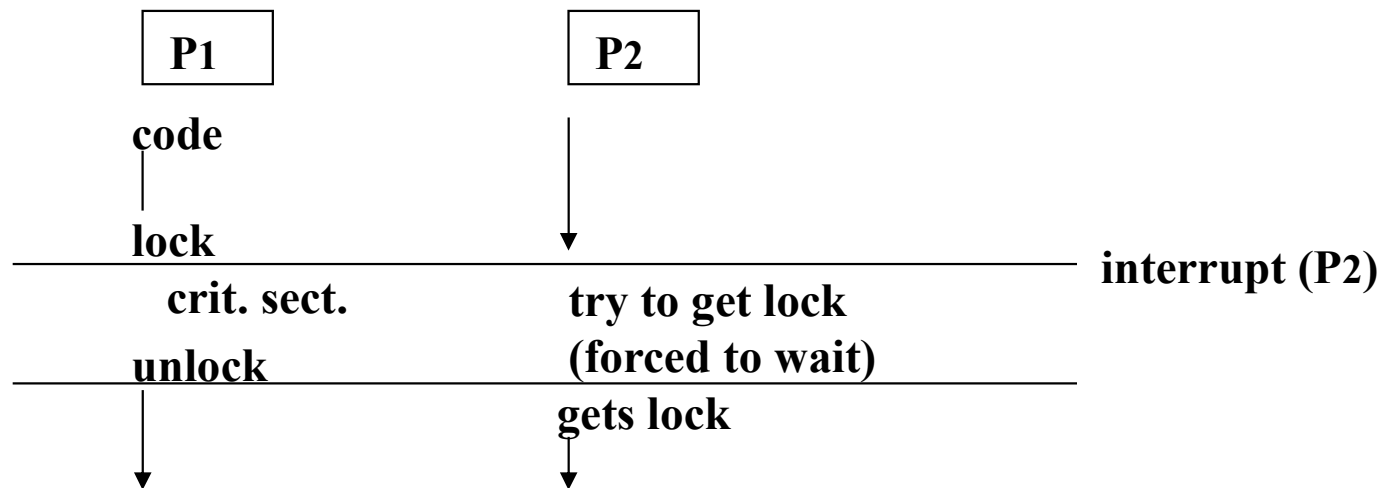
make one attempt; returns 1 on success, 0 on failure

```
void spin_unlock_wait (spinlock_t* lock);
```

wait until available (race condition again!)

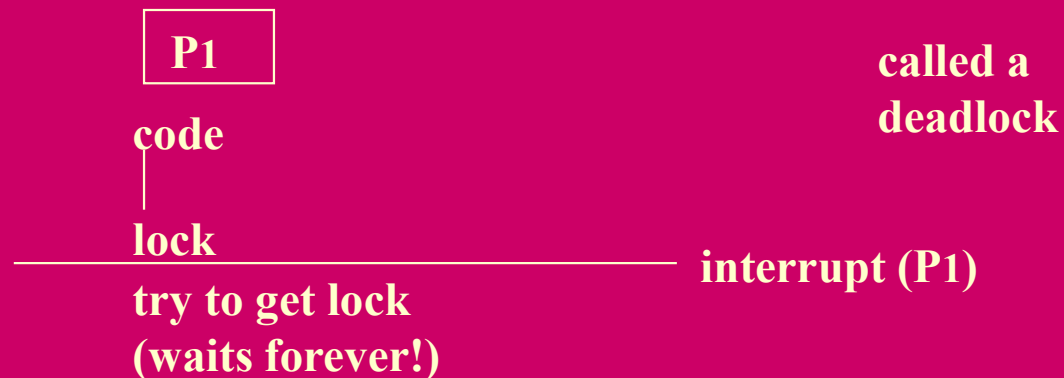
Linux Spin Lock API (cont.)

- Is spinlock enough to protect a critical section?



Linux Spin Lock API (cont.)

- What about ?

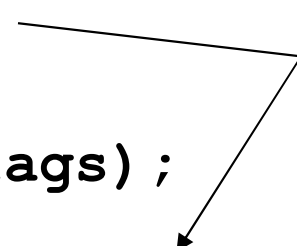


- Still need CLI/STI
- Which is first, CLI or lock?
 - CLI first
 - interrupt may occur between them, leading to scenario above

Linux' Lock/CLI Combo

```
static spinlock_t the_lock = SPIN_LOCK_UNLOCKED;  
unsigned long flags;
```

```
spin_lock_irqsave (&the_lock, flags);  
/* the critical section */  
spin_unlock_irqrestore (&the_lock, flags);
```



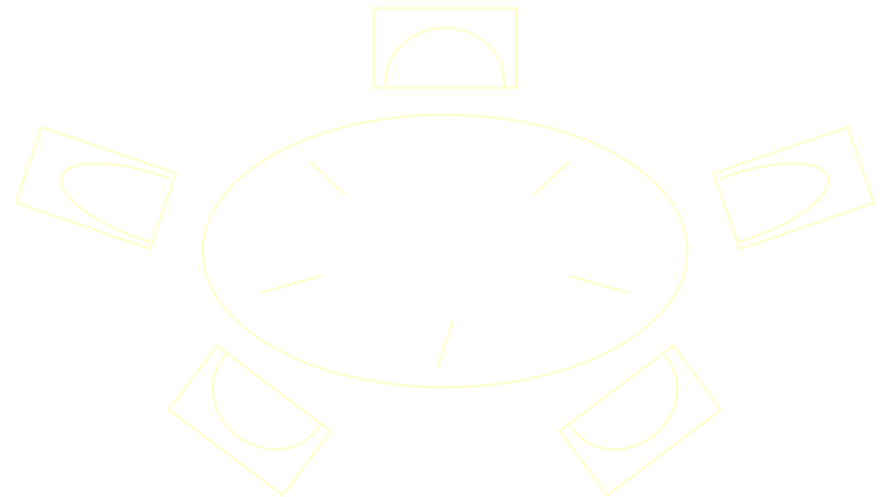
```
asm volatile ("  
    PUSHFL  
    POPL %0  
    CLI  
" : "=g" (flags) /* outputs */  
    : /* inputs */  
    : "memory" /* clobbers */  
);  
spin_lock (&the_lock);
```

Linux' Lock/CLI Combo (cont)

```
spin_unlock (&the_lock);  
asm volatile ("  
    PUSHL %0  
    POPFL  
  
    " : /* outputs */  
    : "g" (flags) /* inputs */  
    : "memory", "cc" /* clobbers */  
);
```

Comments on code

- Notice that *spin_lock_irqsave* changes the flags argument (it's a macro)
- The “memory” argument
 - tells compiler that all memory is written by assembly block
 - prevents compiler from moving memory ops across assembly
- The “cc” argument
 - condition codes change
 - can lead to subtle bugs if left out!
- *spin_lock* and *spin_unlock* calls
 - become NOPs on uniprocessors
 - in that case, calls just change IF
- Restore rationale: may have had IF=0 on entry; if so, STI is unsafe



Summary

- IF=0 protects critical sections from interrupts
 - Prevents interrupts from executing
- On multiprocessor system, we also need to worry about what other processors are doing
 - Use spinlocks to protect critical sections
 - Wait for other CPUs to release lock

Where Do We Put What?

- spin_is_locked
- spin_lock
- spin_unlock
- spin_unlock_wait
- spin_lock_irqsave
- spin_unlock_irqrestore

```
void rxdata(void) {
    int tpos = (rxhpos + rxcnt) % 4;
    if (rxcnt < 4) {
        // read byte from RBR
        rxbuf[tpos] = inb(0x3F8);
        rxcnt = rxcnt + 1;
    } else {
        // receiver buffer overflow
    }
}

char getchar(void) {
    char c;
    while (rxcnt == 0) {
        // wait for data to become available
    }

    c = rxbuf[rxhpos];
    rxhpos = (rxhpos + 1) % 4;
    rxcnt = rxcnt - 1;

    return c;
}
```